

WALLET VIBE — ARCHITECTURE & FEATURE MATRIX

Unified Web, PWA & Native Play Store (.aab) Ecosystem Technical Specification

1. Executive Overview & Multiplatform Vision

WalletVibe is architected as an offline-first, multi-platform personal finance management suite. The system guarantees 100% functional, visual, and performance parity across three deployment targets: (1) **Web Application** (hosted on Netlify CDN), (2) **Progressive Web App (PWA)** (installable via browser prompts), and (3) **Google Play Store Native Android App Bundle (.aab)** via Trusted Web Activity (TWA) architecture. All data transformations, bank description normalizations, and Firestore operations maintain deterministic state regardless of platform.

2. Broader to Narrower Feature Hierarchy

Feature Area	Broader Domain Capability	Narrower Technical Implementation
Expense & Income Tracking	Comprehensive record-keeping for daily expenses, payments, categories, and custom notes.	Real-time balance calculations, category statistics, receipt attachments, PDF export, and instant WhatsApp summary formatting.
Lending & Borrowing	Person-wise debt ledger tracking who owes money and outstanding receivables.	Settlement status toggles, person-specific ledger breakdown, contact phone/email linking, and direct reminder messaging.
Smart Bank Statement Parser	Automated extraction of bank statements from CSV and PDF files across major Indian banks.	Regex description tokenizer (UPI RRN, IMPS, NEFT, CWDR, POS), debit/credit separator, balance tracking, and draft local cache persistence.
Automated Duplicate Cleaner	Intelligent scanner to identify and safely purge duplicate bank records.	Deterministic 5-step decision engine (Amount, DR/CR, Date <= 1d, 12-digit RRN match, Merchant tokens), React Portal virtualized cluster view (<2ms batch), and Firestore writeBatch bulk deletion.
Offline-First Cache Engine	Uninterrupted functionality when internet connectivity is intermittent or unavailable.	Dual-layer caching (in-memory Map + user-isolated localStorage snapshots), fallback queries for legacy UID schemas, and pending queue sync.
Admin Control & Subscriptions	Tiered user subscriptions, registration limits, and administrative controls.	Razorpay integration, trial claim workflows, subscriber capacity limits, duplicate account purger, and remote app configuration toggles.

3. Technical Architecture & Cloud Topology

- **Frontend Core:** Built on React 18 and Vite with vanilla CSS design system, dark mode support, custom modal portal layers, and mobile-first responsive breakpoints (360px to 4K).
- **Data Layer:** Google Cloud Firestore utilizing dual-field querying (userId & uid), batch operations (writeBatch), and strict rule-based security scoping.
- **Cloud Infrastructure:** Hosted on Netlify CDN Edge nodes with Netlify Functions for serverless backend tasks, asset caching, and HTTPS SSL enforcement.

- **Native Wrapper Layer:** Digital Asset Links (assetlinks.json) hosted at /.well-known/assetlinks.json to enable full-screen Trusted Web Activity (TWA) mode in Google Play Store builds.

4. Security & User Error Sanitization

- **Error Sanitizer (userFriendlyError.js):** All raw Firebase exceptions, stack traces, and network timeouts are sanitized into polite, subscriber-friendly guidance messages.
- **Custom Dialog System (CustomDialogModal.jsx):** Native browser alert() and confirm() popups are replaced with portal-rendered glassmorphic dialogs centered in the viewport.
- **Firestore Rules (firestore.rules):** Strict rule scoping enforcing document ownership validation for expenses, lending, bankTransactions, and subscriptions.